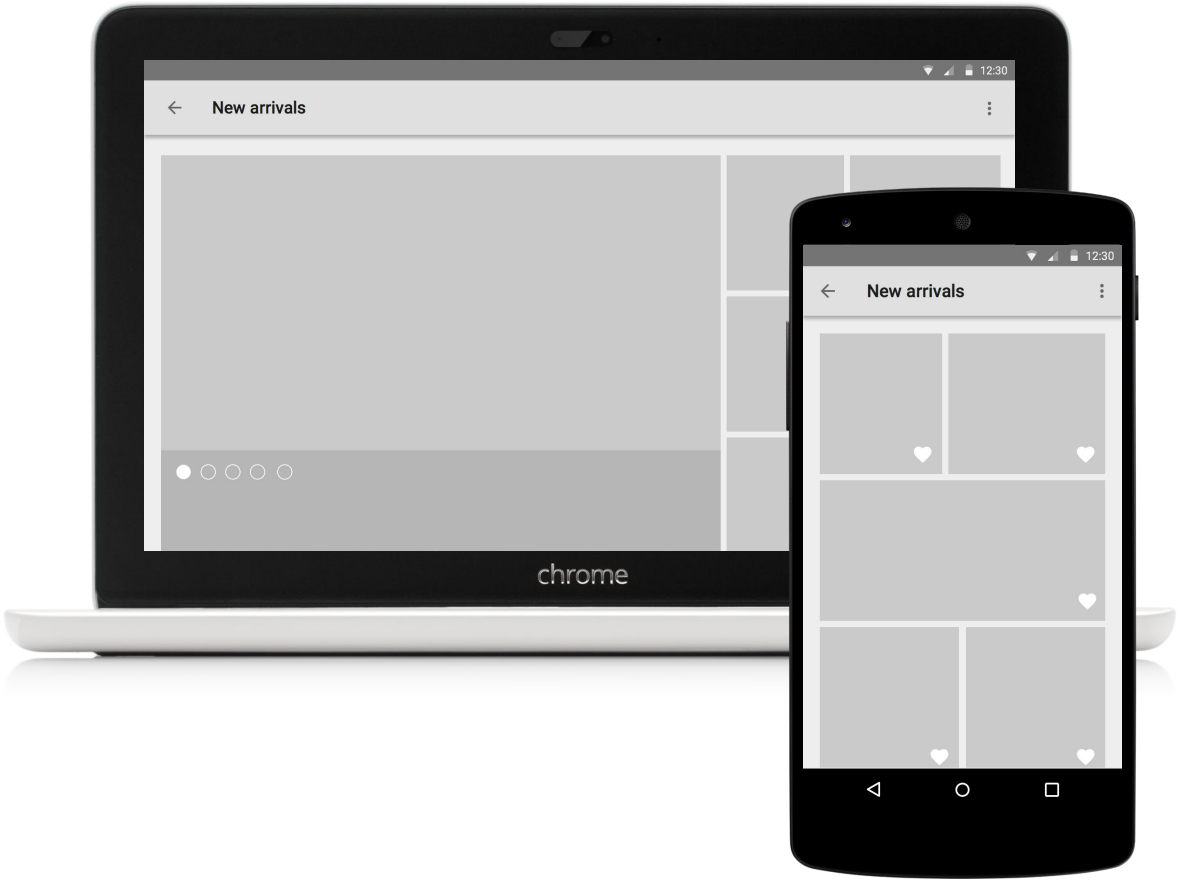
Responsive Design Principles

BCA III

Introduction

- Increasing mobile world
- Ensuring your website looks and functions seamlessly across various devices.
- A responsive website seamlessly adapts to different screen sizes.
- Ensuring optimal user experience across devices such as desktops, laptops, tablets, and smartphones.



Principles of Responsive Web Design

1-Fluid and Grid Layout

1. Instead of using fixed pixel-based grids, RD use relative units like percentages or ems.
2. It allows elements to scale smoothly across different screen sizes.

2-Flexible Images

- images are sized using relative units
- Image should not lose quality or creating layout issues

3-Create layouts and elements that adapt to different screen sizes

Principles of Responsive Web Design

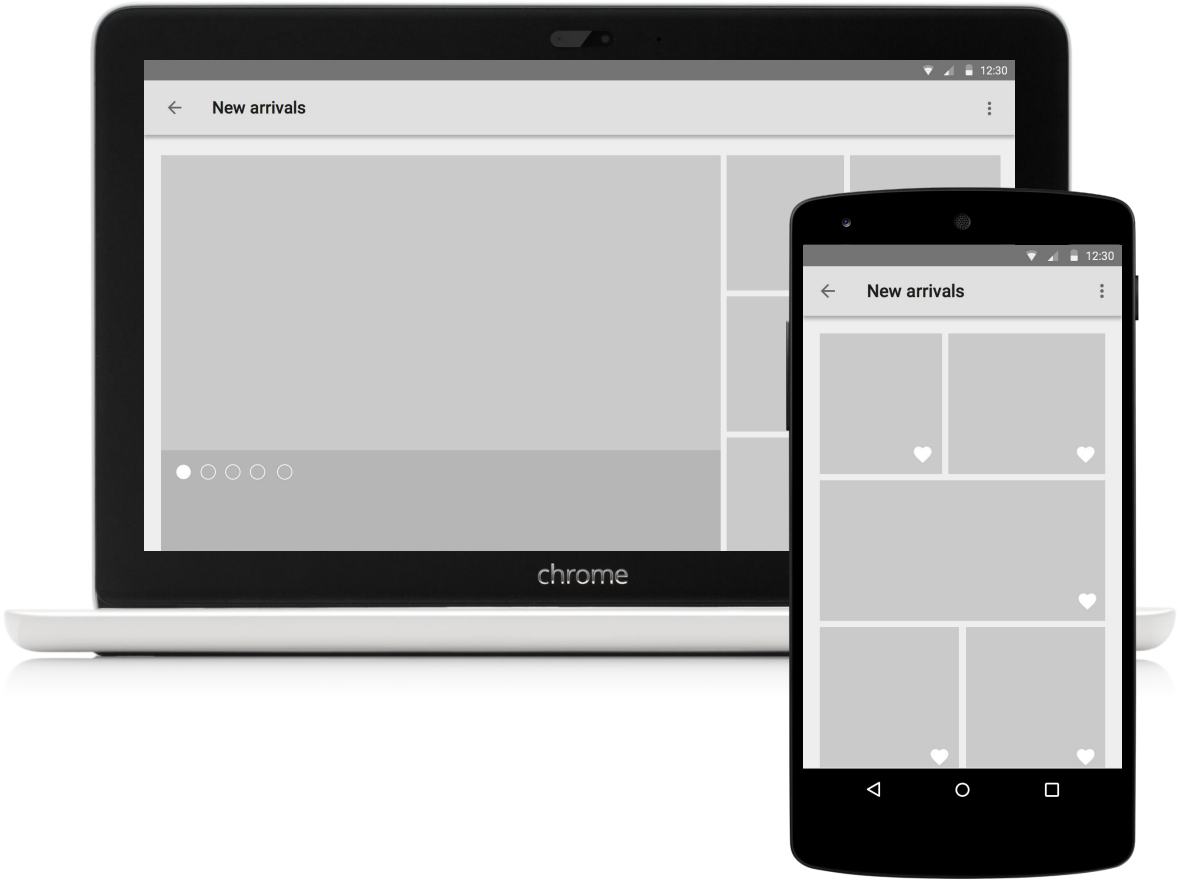
3-Create layouts and elements that adapt to different screen sizes

- Percentage
- Viewport width and VH
- Min-width and max-width
- Em and rem
-

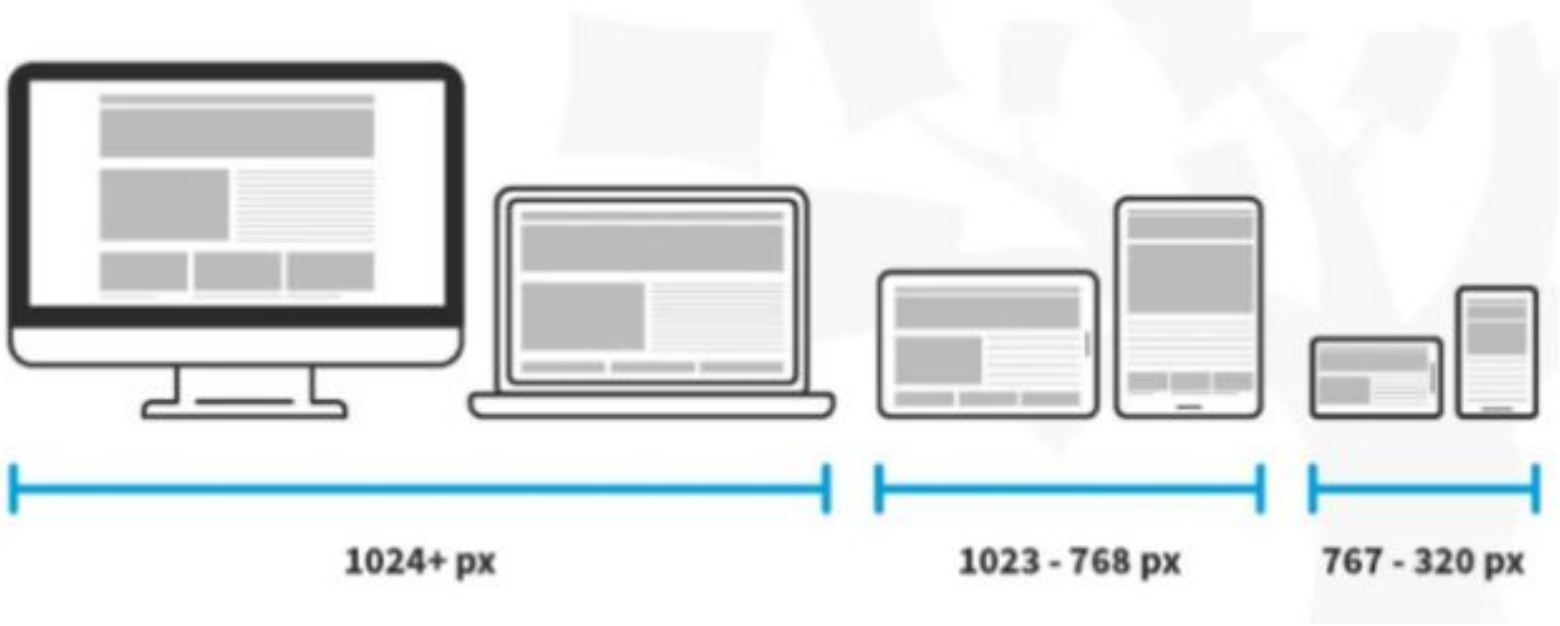
Creating Responsive website: best practices

Best Practices

- Mobile first approach
- Use CSS frameworks
- Responsive units
- Scalable Typography
- Media Queries
- Viewport meta tags
- Flexible images
- User experiance



Working with different screensize



Media Queries in CSS

When the screen (browser window) gets smaller than 768px, element with col class is 100%:

```
@media only screen and (max-width: 768px) {

  .col{

    width: 100%;

  }

}
```

What is SASS?

- Sass stands for Syntactically Awesome Style Sheets.
- It's a preprocessor scripting language that compiles into CSS.
- Sass enhances CSS by providing features like variables, nesting, mixins, and more.
- It simplifies the process of writing CSS and improves code organization and maintainability.
- Sass helps developers write cleaner, more efficient stylesheets, ultimately boosting productivity in web development projects.

Why Use Sass?

- Stylesheets are getting larger, more complex, and harder to maintain. This is where a CSS pre-processor can help.
- Sass lets you use features that do not exist in CSS, like variables, nested rules, mixins, imports, inheritance, built-in functions, and other stuff.

Advantages of scss

- **Cleaner Code:** Sass allows for more organized and modular code through features like nesting and partials, leading to cleaner and more maintainable stylesheets.
- **Variables:** With Sass, you can declare variables to store and reuse values throughout your stylesheets, making it easier to update styles consistently.
- **Nested Syntax:** Sass allows for nesting CSS rules within one another, mirroring the HTML structure and improving readability and maintainability.
- **Mixins:** Sass mixins enable the reuse of CSS declarations, reducing redundancy and promoting a DRY (Don't Repeat Yourself) coding approach.
- **Extends/Inheritance:** The @extend directive in Sass allows styles to inherit from one another, promoting code reuse and reducing duplication.
- **Functions and Operations:** Sass provides built-in functions and supports mathematical operations, enabling more dynamic and expressive stylesheets.
- **Modularity:** Sass supports the concept of partials, allowing stylesheets to be broken down into smaller, manageable files and imported as needed.
- **Community and Ecosystem:** Sass has a large and active community, with extensive documentation, frameworks, and tools available to enhance development workflows.

How Does Sass Work?

- A browser does not understand Sass code. Therefore, you will need a Sass pre-processor to convert Sass code into standard CSS.
- This process is called transpiling. So, you need to give a transpiler (some kind of program) some Sass code and then get some CSS code back.

Thank You

NAST